Now the left neighbor, m_l, steals probability mass from its own left neighbor at index 20, but it steals less because we multiply by γ^2. The right neighbor, m_r, does the same by stealing from its neighbor on the right. Each element in turn steals from either its left or right neighbor until we get to the end of the array. If gamma is close to 1, like 0.99, a lot of probability mass will be redistributed to the support close to r_t.

Let's test our distribution update function. We'll give it an observed reward of –1 starting from a uniform distribution.

Listing 7.3. Redistributing probability mass after a single observation

```
ob_reward = -1
Z = torch.from_numpy(probs).float()
Z = update_dist(ob_reward,torch.from_numpy(support).float(), \
Z,lim=(vmin,vmax),gamma=0.1)
plt.bar(support,Z)
```

You can see in figure 7.12 that the distribution is still fairly uniform, but now there is a distinct "bump" centered at –1. We can control how big this bump is with the discount factor γ. On your own, try changing gamma to see how it changes the update.

Figure 7.12. This is the result of updating an initially uniform probability distribution after observing a single reward. Some probability mass is redistributed toward the support element corresponding to the observed reward.